

Chapter – 2

Object Oriented Principles in Java

Inheritance

Inheritance is an important pillar of OOP(Object-Oriented Programming). It is the mechanism in java by which one class is allowed to inherit the features(fields and methods) of another class. The idea behind inheritance in Java is that you can create new classes that are built upon existing classes. When you inherit from an existing class, you can reuse methods and fields of the parent class. Moreover, you can add new methods and fields in your current class also. Inheritance represents the Is-a relationship which is also known as a parent – child relationship.

Important Terminology

- **Super Class:** The class whose features are inherited is known as superclass(or a base class or a parent class).
- **Sub Class:** The class that inherits the other class is known as a subclass(or a derived class, extended class, or child class). The subclass can add its own fields and methods in addition to the superclass fields and methods.
- **Reusability:** Inheritance supports the concept of “reusability”, i.e. when we want to create a new class and there is already a class that includes some of the code that we want, we can derive our new class from the existing class. By doing this, we are reusing the fields and methods of the existing class.

Is – a relationship and Has – a relationship

Is – a relationship

- Is – a relationship represents inheritance.
- In an Is – a relationship, an object of a subclass can also be treated as an object of its superclass
- For example, a car is a vehicle.

Has - relationship

- Has – a relationship represents composition.
- In a has – a relationship, an object contains as members references to other objects.
- For example, a car has a steering wheel (and a car object has a reference to a steering wheel object).

The syntax of java inheritance

class Subclass – name extends Superclass – name

```
{  
    // methods and fields  
}
```

The extends keyword indicates that you are making a new class that derives from an existing class. The meaning of “extends” is to increase the functionality. In the terminology of java, a class which is inherited is called a parent or super class and the new class is called child or subclass.

Super Class / Base Class / Parent Class

Sub Class / Derived Class / Child Class

Types of Inheritance in java

1. Single Inheritance
2. Multilevel Inheritance
3. Hierarchical Inheritance
4. Multiple Inheritance

1. Single Inheritance

When a class inherits another class, it is known as a single inheritance. In this type of inheritance, there is only one parent class extended by only one child class.

Syntax

Public class A

```
{  
    .....  
}
```

Public class B extends A

```
{  
    .....  
}
```

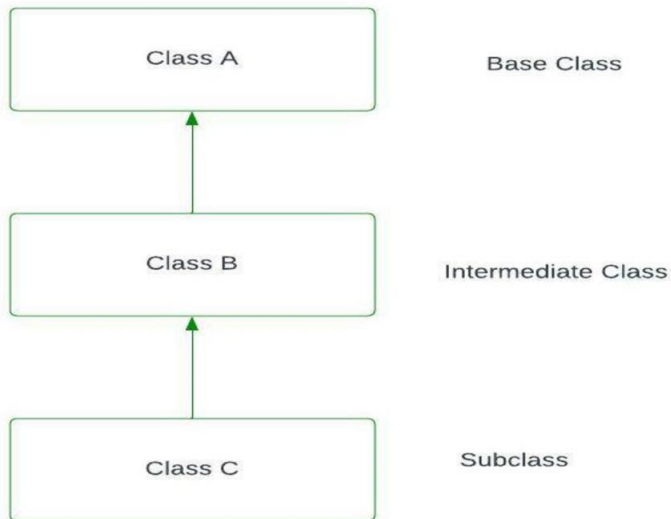
For Example

```
class l
{
    int length;
    void setlength(int l)
    {
        length = l;
    }
}
class B extends l
{
    int breadth;
    void setBreadth(int breadth)
    {
        this.breadth = breadth;
    }
    void area()
    {
        int a = length * breadth;
        System.out.println("Area = " +a);
    }
}

class SingleDemo
{
    public static void main(String args[])
    {
        B ob = new B();
        ob.setlength(5);
        ob.setBreadth(4);
        ob.area();
    }
}
```

Multilevel Inheritance

When there is a chain of inheritance, it is known as multilevel inheritance i.e. a derived class will be inheriting a base class and as well as the derived class also act as the base class to other class. Here class one will be inherited by class two followed by class three. Class two acts as parent class for class three.



Syntax

Public class A

```
{  
    .....  
}
```

Public class B extends A

```
{  
    .....  
}
```

Public class C extends B

```
{  
    .....  
}
```

Example

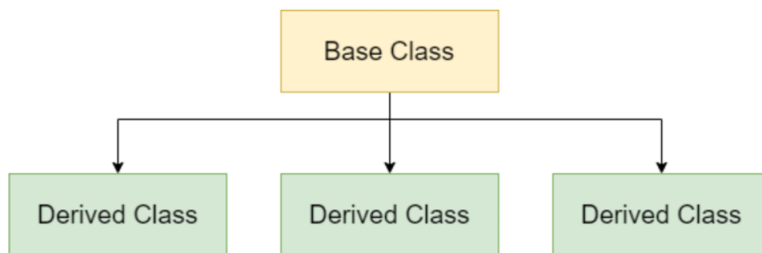
```
class l
{
    int length;
    void setlength(int l)
    {
        length = l;
    }
}
class B extends l
{
    int breadth;
    void setBreadth(int breadth)
    {
        this.breadth = breadth;
    }
    void area()
    {
        int a = length * breadth;
        System.out.println("Area = " +a);
    }
}

class C extends B
{
    int height;

    void setHeight(int height)
    {
        this.height = height;
    }
    void volume()
    {
        int a = length * breadth * height;
        System.out.println("volume = " +a);
    }
}
class MultilevelDemo
{
    Run | Debug
    public static void main(String args[])
    {
        C ob = new C();
        ob.setlength(1:5);
        ob.setBreadth(breadth:4);
        ob.setHeight(height:3);
        ob.area();
        ob.volume();
    }
}
```

Hierarchical Inheritance

When two or more classes inherits a single class, it is known as hierarchical inheritance



Syntax

Public class A

```
{  
    .....  
}
```

Public class B extends A

```
{  
    .....  
}
```

Public class C extends A

```
{  
    .....  
}
```

Example

```
class Circle
{
    double radius;
    void setRadius(double r)
    {
        radius = r;
    }
}

class CircleArea extends Circle
{
    void Area()
    {
        System.out.println("Area = " + 3.14 * radius * radius);
    }
}

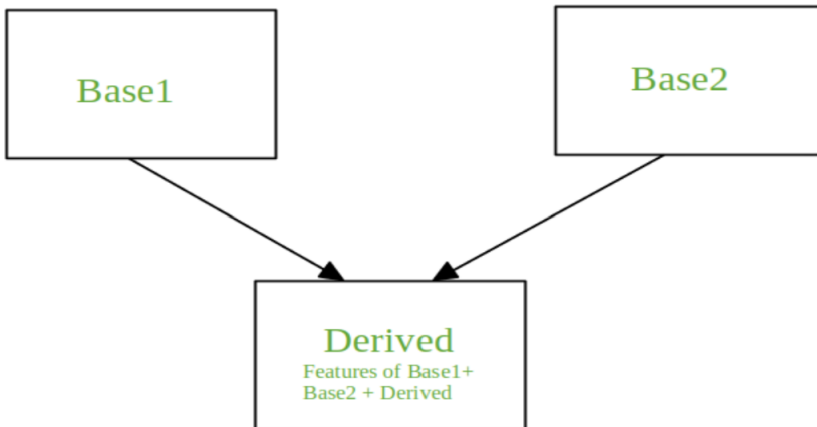
class CirclePerimeter extends Circle
{
    void Perimeter()
    {
        System.out.println("perimeter = " + 2 * 3.14 * radius);
    }
}

class HierarchicalDemo
{
    Run | Debug
    public static void main(String args[])
    {
        CircleArea ob1 = new CircleArea();
        ob1.setRadius(r:3.2);
        ob1.Area();

        CirclePerimeter ob2 = new CirclePerimeter();
        ob2.setRadius(r:5.2);
        ob2.Perimeter();
    }
}
```

Multiple Inheritance

When a class is derived from more than one base class it is called multiple inheritance. The derived class inherits all the features of the base case. To reduce the complexity and simplify the language, multiple inheritance is not supported in java.



Syntax

Public class A

```
{  
    .....  
}
```

Public class B

```
{  
    .....  
}
```

Public class C extends A,B

```
{  
    .....  
}
```


Super Keyword

The Super keyword in java is a reference variable that is used to refer parent class object. The keyword “super” came into the picture with the concept of inheritance. It is used when we want to call the super class variable, method and constructor through sub class object. It is majorly used in the following contexts:

Uses of “Super” keyword

1. To access hidden variable of base class (parent class) from the derived class (child class).(super as an object)

```
class Vehicle
{
    int maxSpeed = 120;
}

class Car extends Vehicle
{
    int maxSpeed = 180;
    void display()
    {
        System.out.println("child Maximum Speed: " +maxSpeed);
        System.out.println("Parent Maximum Speed: " +super.maxSpeed);
    }
}

class superDemo1
{
    Run | Debug
    public static void main(String args[])
    {
        Car ob = new Car();
        ob.display();
    }
}
```

2. To access hidden method of base class (parent class) from the derived class (child class).(super as an object):

```
class Vehicle
{
    void display()
    {
        System.out.println(x:"BMW and Mercedes are the Best Car manufacturer");
    }
}
```

```
class Car extends Vehicle
{
    void display()
    {
        super.display();
        System.out.println(x:"I like Mustang Car");
    }
}
```

```
class superDemo2
{
    Run | Debug
    public static void main(String args[])
    {
        Car ob = new Car();
        ob.display();
    }
}
```

3. To call base class (parent class) constructor from derived class(child class) constructor.(super as a function):

```
class len
{
    int length;
    len(int l)
    {
        length = l;
    }
}
```

```
class B extends len
{
    int breadth;
    B( int l, int b)
    {
        super(l);
        breadth = b;
    }
    void area()
    {
        int a = length * breadth;
        System.out.println(a);
    }
}
```

```
class superDemo3
{
    Run | Debug
    public static void main(String args[])
    {
        B ob = new B(1:5,b:4);
        ob.area();
    }
}
```

Polymorphism

Polymorphism in java is a concept by which we can perform a single action in different ways. Polymorphism is derived from 2 Greek words: poly and morphs. The word “poly” means many and “morphs” means forms.

There are two types of polymorphism in java: Compile Time Polymorphism and Run Time Polymorphism . We can perform polymorphism in java by method overloading and method overriding.

Method Overloading

Method overloading is a feature that allows a class to using a same method name several times but with different method signature (different number or type of arguments and different return type). It is also called compile time polymorphism, early binding, static binding. In order to overload a method, the argument lists of the methods must differ in either of these:

a) Number of Parameter

```
class test
{
    void show(int a, int b)    //2
    {
        s.o.p("1");
    }
    void show(int a)    //1
    {
        s.o.p("2");
    }
    p s v m()
    {
        test t=new test();
        t.show(10,8);
    }
}
```

b) Data type of Parameters

```
class test
{
    void show(int a)
    {
        s.o.p("1");
    }
    void show(string a)
    {
        s.o.p("2");
    }
    p s v m()
    {
        test t=new test();
        t.show("c");
    }
}
```

c) Sequence of Data type of parameters

```
class test
{
    void show(int a, string b)
    {
        s.o.p("1");
    }
    void show(string a, int b)
    {
        s.o.p("2");
    }
    p s v m()
    {
        test t=new test();
        t.show("b",10);
    }
}
```

Example

```
class Test
{
    public void check(String msg)
    {
        System.out.println(msg);
    }

    public void check(int m)
    {
        System.out.println("square of " +m+ " is "+ m * m);
    }

    public double check(double a, double b)
    {
        return a+b;
    }
}

class OverloadDemo
{
    Run | Debug
    public static void main(String args[])
    {
        Test ob = new Test();
        ob.check(msg:"Good Morining Students");
        ob.check(m:5);
        //double result = ob.check(10.2,5.6);
        System.out.println(ob.check(a:10.2,b:5.6));
    }
}
```

Here, the method check() is overloaded 3 times: first method has 1 string parameter, second method has 1 int parameter, third method has 2 double parameter.

Method Overriding

Declaring method in sub class which is already present in parent class is known as method overriding. Overriding is done so that a child class can give its own implementation to a method which is already provided by the parent class. In this case the method in parent class is called overridden method and method in child class is called overriding method.

Method overriding is a feature that allows a class to using a same method name several times with same method signature in base class and derived class. It is also called run time polymorphism, late binding, dynamic binding.

Example

```
class Calculator
{
    public int add(int n1, int n2)
    {
        return n1 + n2;
    }
}

class advance_calculator extends Calculator
{
    public int add(int n1, int n2)
    {
        return n1 + n2 + 1;
    }
}

class overrideDemo
{
    Run | Debug
    public static void main(String[] args)
    {
        advance_calculator calc1 = new advance_calculator();
        int result = calc1.add(n1:5, n2:3);
        System.out.println("The result is: " + result);
        Calculator calc2 = new Calculator();
        System.out.println(calc2.add(n1:2, n2:4));
    }
}
```

Final Keyword

There are three uses of final Keyword

1. To create a constant

Final double PI = 3.14;

2. To create a final method

A method which cannot be overridden.

```
class A
{
    final void test()
    {
        -----
        -----
    }
}
class B extends A
{
    void test()           // not allowed
    {
        -----
        -----
    }
}
```

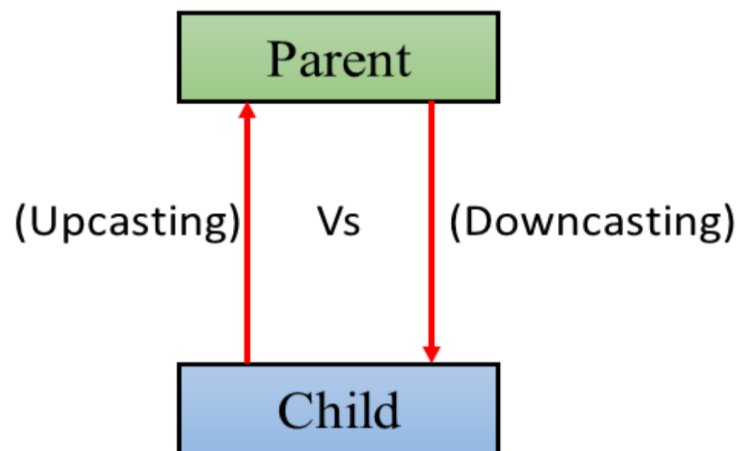

3. To create a final class

A class which cannot be inherited.

```
final class A
{
    -----
    -----
}
class B extends A    // not allowed
{
    -----
    -----
}
```

UpCasting and DownCasting

Typecasting is one of the most important concepts which basically deals with the conversion of one data type to another datatype implicitly or explicitly.



Just like the data types, the objects can also be typecasted. However, in objects, there are only two types of objects, i.e. parent object and child object. Therefore, typecasting of objects basically means that one type of object (i.e.) child or parent to another. There are two types of typecasting. They are:

1. Upcasting

Upcasting is the typecasting of a child object to a parent object. Upcasting can be done implicitly. Upcasting gives us the flexibility to access the parent class members but it is not possible to access all the child class members using this feature. Instead of all the members,

we can access some specified members of the child class. For instance, we can access the overridden methods.

2. Downcasting

Similarly, downcasting means the typecasting of a **parent object to a child object**. Downcasting cannot be implicit.

Example

```
class A
{
    public void display()
    {
        System.out.println(x:"Good Morning");
    }
}
class B extends A
{
    public void display1()
    {
        System.out.println(x:"Good Evening");
    }
}

class CastingDemo
{
    Run | Debug
    public static void main(String[] args)
    {
        A obj = (A) new B (); // Upcasting
        obj.display();

        B obj1 = (B) obj; // Downcasting
        obj1.display1();
    }
}
```

Abstraction in Java

Data Abstraction is the property by virtue of which only the essential details are displayed to the user. The trivial or the non-essentials units are not displayed to the user. Ex: A car is viewed as a car rather than its individual components.

Data Abstraction may also be defined as the process of identifying only the required characteristics of an object ignoring the irrelevant details. The properties and behaviors of an object differentiate it from other objects of similar type and also help in classifying/grouping the objects.

Consider a real-life example of a man driving a car. The man only knows that pressing the accelerators will increase the speed of car or applying brakes will stop the car, but he does not know about how on pressing the accelerator the speed is actually increasing, he does not know about the inner mechanism of the car or the implementation of the accelerator, brakes, etc in the car. This is what abstraction is.

In java, abstraction is achieved by interfaces and abstract classes. We can achieve 100% abstraction using interfaces. A class that is declared using “abstract” keyword is known as abstract class. It may or may not include abstract methods which means in abstract class we can have concrete methods (methods with body) as well along with abstract methods (without an implementation, without braces, and followed by a semicolon). An abstract class can not be instantiated (We are not allowed to create object of Abstract class).

Rules for Abstract Class

- If the class is having few abstract methods and few concrete methods: declare it as abstract class.
- If the class is having only abstract methods: declare it as interface.
- Object creation of abstract class is not allowed
- We cannot declare abstract constructors, or abstract static methods.
- Any subclass of an abstract class must either implement all of the abstract methods in the superclass, or be itself declared abstract.

Specifying abstract keyword before the class during declaration, makes it abstract

Rules for Abstract method

- Abstract method has no body.
- Always end the declaration with a semicolon();
- It must be overridden. An abstract class must be extended and in a same way abstract method must be overridden.
- Abstract method must be in a abstract class.

Note: The class which is extending abstract class must override (or implement) all the abstract methods.

Example:

```
abstract class shape
{
    double dim1, dim2;
    shape(double d1, double d2)
    {
        dim1 = d1;
        dim2 = d2;
    }
    abstract void area();
}

class rectangle extends shape
{
    rectangle(double d1, double d2)
    {
        super(d1,d2);
    }

    void area()
    {
        double a = dim1 * dim2;
        System.out.println("Area of rectangle " +a);
    }
}

class triangle extends shape
{
    triangle(double d1, double d2)
    {
        super(d1,d2);
    }
    void area()
    {
        double a =0.5* dim1 * dim2;
        System.out.println("Area of triangle "+a);
    }
}

class methodOverriding1
{
    public static void main(String args[])
    {
        //shape s = new shape(2.2,1.2);
        //s.area();
        shape r1 = new rectangle(4.5,2.3);
        r1.area();
        shape t1 = new triangle(3.2,4.6);
        t1.area();
    }
}
```

Interface

Interface looks like class but it is not a class. An interface can have methods and variables just like the class but the methods declared in interface are by default abstract (only method signature, no body). Also, the variables declared in an interface are public, static & final by default. Java does not support multiple inheritance i.e. classes in java cannot have more than one superclass.

A large number of real-life application requires the use of multiple inheritance where a class inherits properties and method from several distinct classes. For this purpose, java provides an alternate approach called interface to support the concept of multiple inheritance. A java class cannot be a subclass of more than one superclass, but it can implement more than one interface thus supporting multiple inheritance.

Implementing Interface

Interfaces are normally used as “*superclasses*” whose properties are inherited by classes. Class can implement interface using keyword **implements** as shown below:

Use of Interface to achieve multiple inheritance

Let us say we have two interfaces A & B and two classes C & D. Then we can achieve multiple inheritance using interfaces as follows:

```
interface A{ }
```

```
interface B{ }
```

```
class C{ }
```

```
class D extends C implements A,B { }
```

Example

```
interface interface1{
    void display1();
}

interface interface2{
    void display2();
}

class A{
    void display3() {
        System.out.println("I am inside class");
    }
}

class B extends A implements interface1,interface2{
    public void display1() {
        System.out.println("I am inside interface1");
    }
    public void display2() {
        System.out.println("I am inside interface2");
    }
}

public class MultiLevel {
    public static void main(String[] args) {
        B obj=new B();
        obj.display1();
        obj.display2();
        obj.display3();
    }
}
```

Output

```
I am inside interface1
I am inside interface2
I am inside class
```

Another Example

```
interface College
{
    String c_name = "Everest Engineering College";
    void showInfo();
}

class Student
{
    String s_name ;
    String faculty ;

    Student(String a, String b)
    {
        s_name = a;
        faculty = b;
    }
}

class StudentDetails extends Student implements College
{
    public StudentDetails(String a, String b)
    {
        super(a,b);
    }
    public void showInfo()
    {
        System.out.println("student name: " +s_name);
        System.out.println("college name: " +c_name);
    }
}

class interface1Demo
{
    Run | Debug
    public static void main(String args[])
    {
        StudentDetails p1 = new StudentDetails(a:"Nishanta", b:"BE Computer");
        p1.showInfo();
    }
}
```

Encapsulation in Java

Encapsulation is defined as the wrapping up of data under a single unit. It is the mechanism that binds together code and the data it manipulates. Another way to think about encapsulation is, it is a protective shield that prevents the data from being accessed by the code outside this shield.

- Technically in encapsulation, the variables or data of a class is hidden from any other class and can be accessed only through any member function of its own class in which it is declared.
- As in encapsulation, the data in a class is hidden from other classes using the data hiding concept which is achieved by making the members or methods of a class private, and the class is exposed to the end-user or the world without providing any details behind implementation using the abstraction concept, so it is also known as a combination of data-hiding and abstraction.
- Encapsulation can be achieved by Declaring all the variables in the class as private and writing public methods in the class to set and get the values of variables.

Advantages of Encapsulation

Data Hiding: The user will have no idea about the inner implementation of the class. It will not be visible to the user how the class is storing values in the variables. The user will only know that we are passing the values to a setter method and variables are getting initialized with that value.

Increased Flexibility: We can make the variables of the class as read-only or write-know that only depending on our requirement. If we wish to make the variables read-only then we have to omit the setter methods like setName(), setAge(), etc. from the above program or if we wish to make the variables as write-only then we have to omit the get methods like getName(), getAge(), etc. from the above program.

Reusability: Encapsulation also improves the re-usability and easy to change with new requirements. Testing code is easy: Encapsulated code is easy to test for unit testing

Example

```
class Student1
{
    private String name;
    private int roll;

    public void setName(String n)
    {
        name = n;
    }

    public String getName()
    {
        return name;
    }

    public void setRoll(int r)
    {
        roll = r;
    }

    public int getRoll()
    {
        return roll;
    }
}

class StudentDemo1
{
    Run | Debug
    public static void main(String[] args)
    {
        Student1 ob = new Student1(); // creating an instance of the Student1 class
        ob.setName(n:"Nishanta"); // calling setName method using the object
        ob.setRoll(r:12); // calling setRoll method using the object

        Student1 ob1 = new Student1();
        ob1.setName(n:"Ajit");
        ob1.setRoll(r:4);

        System.out.println("Name: " + ob.getName());
        System.out.println("Roll: " + ob.getRoll());

        System.out.println("Name: " + ob1.getName());
        System.out.println("Roll_no: " + ob1.getRoll());
    }
}
```

Inner Class

It is possible to define a class within another class; such classes are known as nested classes. The scope of a nested class is bounded by the scope of its enclosing class. Thus, if class B is defined within class A, then B does not exist independently of A. A nested class has access to the members, including private members, of the class in which it is nested. However, the enclosing class does not have access to the members of the nested class. A nested class that is declared directly within its enclosing class scope is a member of its enclosing class. It is also possible to declare a nested class that is local to a block.

There are two types of nested classes: static and non-static. A static nested class is one that has the static modifier applied. Because it is static, it must access the members of its enclosing class through an object. That is, it cannot refer to members of its enclosing class directly. Because of this restriction, static nested classes are seldom used.

The most important type of nested class is the inner class. An inner class is a non-static nested class. It has access to all of the variables and methods of its outer class and may refer to them directly in the same way that other non-static members of the outer class do.

Example

```
class college
{
    String c_name ="Nist college";
    String location = "Gwarko";

    public void show()
    {
        System.out.println(c_name + " is located at " + location );
    }

    class student
    {
        String s_name = "cadet" ;
        int roll_no = 9;

        public void display()
        {
            System.out.println(s_name+ " roll number is " +roll_no);
        }
    }
}

class innerDemo
{
    Run | Debug
    public static void main (String args[])
    {
        college ob1 = new college();
        ob1.show();
        college.student ob2 = ob1.new student();
        ob2.display();
    }
}
```

Wrapper Class

A wrapper class is a class whose object wraps or contains a primitive data types. When we create an object to a wrapper class, it contains a field and in this field, we can store a primitive data types. In other word, we can wrap a primitive value into a wrapper class object. Wrapper classes are needed for following reasons..

- They convert primitive data types into objects. Objects are needed if we wish to modify the arguments passed into a method (because primitive data types are passed by value).
- The class in java.util package handles only objects and hence wrapper classes help in thos case also.
- Data structures in the collection framework, such as ArrayList and Vector, store only objects(reference types) and primitive types.
- An object is needed to support synchronization in multithreading

Primitive Data types and their corresponding Wrapper Class

Primitive Data type	Wrapper Class
char	Character
byte	Byte
short	Short
long	Integer
float	Float
double	Double
boolean	Boolean

Auto – Boxing

- Automatic conversion of primitive types to the object of their corresponding wrapper classes is known as autoboxing.
- For example, conversion of int to Integer, Long to Long, double to Double

Auto - Unboxing

- It is just the reverse process of autoboxing. Automatically, converting an object of a wrapper class to its corresponding primitive type as unboxing. For example , conversion Integer to int, Long to long, Double to double etc.

Example

```
public class Wrapper
{
    public static void main(String args[])
    {
        int num = 7;
        //Integer num1 = new Integer(8);

        //Integer num1 = new Integer(num); //boxing
        Integer num1 = num; // auto -boxing
        System.out.println(num1);

        //int num2 = num1.intValue(); //un-boxing
        int num2 = num1; // auto-unboxing
        System.out.println(num2);

        String str = "456123";
        int num3 = Integer.parseInt(str);
        System.out.println(num3);

        float num5 = 18.6f;
        Float result = num5;
        System.out.println(result);

        float result1 = result.floatValue();
        //float result1 = result;
        System.out.println(result1);
    }
}
```